

Flutter Transmit Security Authentication - User Guide

Overview

The Flutter Transmit Security Authentication module provides secure authentication methods including PIN codes, biometrics, and WebAuthn for Flutter applications. This guide covers installation, setup, and usage.

Note: For additional integration guides such as configuring passkeys and advanced platform-specific features, we recommend reading the native SDK documentation:

- [iOS documentation](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios/)
(https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios/)
- [Android documentation](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android/#Con)
(https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android/#Con
[line](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android/#Con))

Installation

1. Add to pubspec.yaml

```
dependencies:  
  flutter_ts_authentication:  
    git:  
      url: https://github.com/TransmitSecurity/flutter\_ts\_authentication.git  
      ref: 0.0.3 # Use the latest version tag
```

2. Install dependencies

```
flutter pub get
```

Platform Setup

iOS Setup

1. Minimum Requirements

- iOS 15.0+
- Xcode 14+

Note on the iOS floor. The plugin declares iOS 15.0 in both its Package.swift and its podspec. The underlying native Authentication SDK declares a lower floor (iOS 13 as of 1.2.2, corrected from iOS 15 in 1.2.1), but the plugin deliberately keeps 15.0: nothing in the plugin is tested on iOS 13/14, and a stricter consumer constraint is always valid. If you need iOS 13/14 support, raise it with Transmit Security rather than editing the constraint locally.

2. Update ios/Runner/Info.plist

Add biometric permission:

```
<key>NSFaceIDUsageDescription</key>
<string>This app uses Face ID for secure authentication</string>
```

Note: This module uses Swift Package Manager (SPM) for iOS dependencies. No additional CocoaPods setup is required.

Android Setup

1. Minimum Requirements

- Android API 23+
- compileSdkVersion 34+

2. Add Maven repository to android/build.gradle

```
allprojects {
    repositories {
        google()
        mavenCentral()
        maven {
            url "https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/"
        }
    }
}
```

3. Update android/app/build.gradle

```
android {  
    compileSdkVersion 34  
  
    defaultConfig {  
        minSdkVersion 23  
        targetSdkVersion 34  
    }  
}
```

4. Add permissions to android/app/src/main/AndroidManifest.xml

```
<uses-permission android:name="android.permission.USE_BIOMETRIC" />  
<uses-permission android:name="android.permission.USE_FINGERPRINT" />  
<uses-permission android:name="android.permission.INTERNET" />
```

5. (Optional) Configure biometric prompt strings in android/app/src/main/res/values/strings.xml

```
<string name="BiometricPromptTitle">Authenticate with Biometrics</string>  
<string name="BiometricPromptSubtitle">Use your device biometrics to  
authenticate.</string>  
<string name="BiometricPromptCancel">Cancel</string>
```

These strings customize the biometric authentication dialog shown to users.

Integration

Important Setup Requirements

Maven Repository Configuration

Add the Maven repository in **both** the module-level and app-level build.gradle files:

Module-level android/build.gradle:

```
allprojects {  
    repositories {  
        google()  
        mavenCentral()  
        maven {  
            url "https://transmit.jfrog.io/artifactory/transmit-security-  
gradle-release-local/"  
        }  
    }  
}
```

App-level android/app/build.gradle:

```
repositories {
  google()
  mavenCentral()
  maven {
    url "https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/"
  }
}
```

Android MainActivity Configuration

IMPORTANT: Make sure that your MainActivity inherits from FlutterFragmentActivity:

```
import io.flutter.embedding.android.FlutterFragmentActivity

class MainActivity: FlutterFragmentActivity() {
  // Your activity code
}
```

Why FlutterFragmentActivity?

- Required for biometric authentication features
- Provides proper fragment management for native biometric dialogs
- Ensures compatibility with Android's biometric prompt APIs

Basic Usage

1. Import the module

```
import 'package:flutter_ts_authentication/flutter_ts_authentication.dart';
```

2. Initialize the SDK

```
final auth = FlutterTsAuthentication();

// Initialize SDK
await auth.initializeSDK();

// Initialize with configuration
final initOptions = TSEInitOptions(
  webAuthnInitOptions: TSEWebAuthnInitOptions(
    startAuthentication: '/auth/webauthn/authenticate',
    startRegistration: '/auth/webauthn/register',
  ),
);

await auth.initialize(
  'your-client-id',
  'your-domain.com',
  'https://api.your-domain.com',
  initOptions
);
```

Initialization Parameters

- **clientId**: Your Transmit Security client ID
- **domain**: Your application domain
- **baseUrl**: API endpoint URL (default: <https://api.transmitsecurity.io>)
- **initOptions**: Configuration options containing WebAuthn settings
 - **webAuthnInitOptions**: WebAuthn-specific configuration
 - **rpld**: Relying Party ID (usually your domain)
 - **rpName**: Human-readable name for your application

API Reference

PIN Code Authentication

Register PIN Code

```
try {
  final result = await auth.registerPinCode('username', '123456');
  print('Public Key ID: ${result.publicKeyId}');
  print('Context ID: ${result.contextIdentifier}');

  // Commit the registration
  await auth.commitPinRegistration(result.contextIdentifier);
} catch (e) {
  print('PIN registration failed: $e');
}
```

Authenticate with PIN Code

```
try {
  final result = await auth.authenticatePinCode(
    'username',
    '123456',
    'challenge-string'
  );
  print('Authenticated! Public Key ID: ${result.publicKeyId}');
  print('Signature: ${result.signature}');
} catch (e) {
  print('PIN authentication failed: $e');
}
```

Unregister PIN Code

```
try {
    final result = await auth.unregisterPinCode('username');
    print('Public Key ID: ${result.publicKeyId}');

    // Commit the unregistration once your backend acknowledged it
    await auth.commitPinUnregistration(result.contextIdentifier);
} catch (e) {
    print('PIN unregistration failed: $e');
}
```

Biometric Authentication

Check Biometrics Availability

```
final status = await auth.nativeBiometricsStatus();
if (status == TSBiometricsStatus.available) {
    final type = await auth.nativeBiometricsType();
    print('Biometrics available: $type');
}
```

TSBiometricsStatus values: available, notEnrolled, notAvailable, permissionDenied (iOS), lockedOut (iOS), securityUpdateRequired (Android), unsupported (Android), unknown.

TSBiometricsType values: faceId, touchId, opticId, none, and biometric — reported on Android, which does not expose the concrete modality.

Register Biometrics

```
try {
    final result = await auth.registerNativeBiometrics('username');
    print('Biometric registered: ${result.publicKeyId}');
} catch (e) {
    print('Biometric registration failed: $e');
}
```

Unregister Biometrics

```
try {
    final result = await auth.unregisterNativeBiometrics('userId');
    print('Biometric unregistered: ${result.publicKeyId}');
} catch (e) {
    print('Biometric unregistration failed: $e');
}
```

Authenticate with Biometrics

```
try {
    final result = await auth.authenticateNativeBiometrics(
        'username',
        'challenge-string'
    );
    print('Biometric auth successful: ${result.signature}');
} catch (e) {
    print('Biometric authentication failed: $e');
}
```

Approval with Biometrics

```
try {
    final result = await auth.approvalNativeBiometrics(
        'username',
        'challenge-string'
    );
    print('Approval successful: ${result.signature}');
} catch (e) {
    print('Biometric approval failed: $e');
}
```

WebAuthn

Check WebAuthn Support

```
final isSupported = await auth.isWebAuthnSupported();
if (isSupported) {
    // Proceed with WebAuthn operations
} else {
    // Use alternative authentication method
}
```

Register WebAuthn

```
try {
    final result = await auth.registerWebAuthn('username', 'Display Name');
    print('WebAuthn registered: ${result.result}');
} catch (e) {
    print('WebAuthn registration failed: $e');
}
```

Authenticate with WebAuthn

```
try {
    final result = await auth.authenticateWebAuthn('username');
    print('WebAuthn auth successful: ${result.result}');
} catch (e) {
    print('WebAuthn authentication failed: $e');
}
```

WebAuthn Transaction Signing

```
try {
  final result = await auth.signWebauthnTransaction('username');
  print('Transaction signed: ${result.result}');
} catch (e) {
  print('Transaction signing failed: $e');
}
```

WebAuthn Options

The options parameter on `approvalWebAuthn`, `approvalWebAuthnWithData`, `authenticateWebAuthnWithData` and `signWebauthnTransactionWithData` accepts the names in

`TSWebAuthnAuthenticationOptionValues`:

Option	Effect
<code>preferLocalCredentials</code>	Prefer credentials already present on the device over server-provided ones. iOS only.

Platform behavior. `options` is honored on **iOS only** — the native iOS SDK accepts a `WebAuthnAuthenticationOptions` set, while the native Android SDK's equivalent methods declare no

`options` parameter. On Android the names are still **validated** so that an unrecognized value fails identically on both platforms, and are then a documented no-op.

An unrecognized option name fails rather than being silently ignored. Pass an empty list for default behavior. The failure arrives as a `PlatformException` with code ==

`'invalidArguments'`,

and its details decode to `TSAuthenticationErrorCode.invalidArguments`:

```
try {
  await auth.authenticateWebAuthnWithData(authData, <String>['typo-here']);
} on PlatformException catch (e) {
  final details = TSAuthenticationErrorDetails.fromPlatformException(e);
  if (details.code == TSAuthenticationErrorCode.invalidArguments) {
    // details.description names the offending values and the supported set
    print(details.description);
  }
}
```

WebAuthn Approval

```
// Method 1: With approval data
final approvalData = {'transaction': 'transfer', 'amount': '100'};

// Pass an empty list for default behavior, or use the named constants.
final options = <String>[
    TSWebAuthnAuthenticationOptionValues.preferLocalCredentials,
];

try {
    final result = await auth.approvalWebAuthn(
        approvalData,
        'username',
        options
    );
    print('Approval successful: ${result.result}');
} catch (e) {
    print('WebAuthn approval failed: $e');
}

// Method 2: With raw authentication data
final authData = TSWebAuthnAuthenticationData(data: {
    'webauthnSessionId': 'session-id',
    'credentialRequestOptions': {}
});

try {
    final result = await auth.approvalWebAuthnWithData(authData, options);
    print('Approval successful: ${result.result}');
} catch (e) {
    print('WebAuthn approval failed: $e');
}
```

WebAuthn with server-provided data

When your backend performs the `start registration` / `start authentication` call itself, pass its response straight through instead of letting the SDK make the call:

```

final registrationData = TSWebAuthnRegistrationData(data: {
  'webauthnSessionId': 'session-id',
  'credentialCreationOptions': { /* rp, user, challenge, pubKeyCredParams, ...
*/ },
});
final registration = await auth.registerWebAuthnWithData(registrationData);

final authData = TSWebAuthnAuthenticationData(data: {
  'webauthnSessionId': 'session-id',
  'credentialRequestOptions': { /* challenge, allowCredentials, rpId, ... */ },
});
final authentication = await auth.authenticateWebAuthnWithData(authData,
options);
final signature = await auth.signWebauthnTransactionWithData(authData,
options);

```

Device Information

```

try {
  final deviceInfo = await auth.getDeviceInfo();
  print('Device Public Key ID: ${deviceInfo.publicKeyId}');
  print('Device Public Key: ${deviceInfo.publicKey}');
} catch (e) {
  print('Failed to get device info: $e');
}

```

Logging Configuration

Enable/Disable Logging

```

try {
  // Enable detailed logging for debugging
  final success = await auth.setLoggingEnabled(true);
  if (success) {
    print('Logging enabled successfully');
  }

  // Disable logging for production
  final disabled = await auth.setLoggingEnabled(false);
  if (disabled) {
    print('Logging disabled successfully');
  }
} catch (e) {
  print('Failed to configure logging: $e');
}

```

Note: Logging configuration helps with debugging authentication issues during development. It's recommended to disable logging in production builds for security and performance reasons.

TOTP (Time-based One-Time Password)

Register TOTP

```
try {
    final result = await auth.registerTOTP(
        'otpauth://totp/Example:user@example.com?
secret=JBSWY3DPEHPK3PXP&issuer=Example',
        TSTOTPSecurityType.biometric // or TSTOTPSecurityType.none
    );

    print('TOTP registered successfully');
    print('Issuer: ${result.issuer}');
    print('Label: ${result.label}');
    print('UUID: ${result.uuid}');
} catch (e) {
    print('TOTP registration failed: $e');
}
```

Security Types:

- TSTOTPSecurityType.biometric: Requires biometric authentication to generate codes
- TSTOTPSecurityType.none: No additional security required
- TSTOTPSecurityType.devicePin: Requires the device PIN / passcode
- TSTOTPSecurityType.devicePinOrBiometric: Requires the device PIN / passcode or biometrics

devicePin and devicePinOrBiometric require native Authentication SDK Android 1.0.30 / iOS 1.2.2

or later. When the device cannot satisfy the requested protection, registration fails with TSPAuthenticationErrorCode.devicePinNotAvailable or TSPAuthenticationErrorCode.devicePinOrBiometricNotAvailable.

Generate TOTP Code

```
try {
    final result = await auth.generateTOTPCode('your-totp-uuid');
    print('Generated TOTP code: ${result.code}');
} catch (e) {
    print('Failed to generate TOTP code: $e');
}
```

Generate TOTP Code with Challenge

```
try {
    final result = await auth.generateTOTPCodeWithChallenge(
        'your-totp-uuid',
        'challenge-string'
    );
    print('Generated TOTP code with challenge: ${result.code}');
} catch (e) {
    print('Failed to generate TOTP code with challenge: $e');
}
```

Use Cases:

- **Basic TOTP:** Use generateTOTPCode() for standard time-based authentication

- **Challenge-based TOTP:** Use `generateTOTPCodeWithChallenge()` for transaction signing or enhanced security scenarios

Device Key Signing

Sign challenges using device keys for secure authentication and transaction validation:

```
try {
    final TSSignChallengeResult result = await auth.signWithDeviceKey('challenge-to-sign');
    print('Signature: ${result.signature}');
} catch (e) {
    print('Device key signing failed: $e');
}
```

Use Cases:

- **Transaction Signing:** Sign transaction data for secure approval workflows
- **Challenge-Response:** Implement challenge-response authentication protocols
- **Device Authentication:** Prove device identity using cryptographic signatures
- **Secure Communications:** Sign messages for end-to-end security verification

Error Handling

All methods throw exceptions on failure. Wrap calls in try-catch blocks:

```
try {
    final result = await auth.authenticatePinCode('user', '1234', 'challenge');
    // Success
} on PlatformException catch (e) {
    switch (e.code) {
        case 'INVALID_ARGUMENTS':
            print('Invalid parameters provided');
            break;
        case 'NULL_RESULT':
            print('No result returned from native platform');
            break;
        default:
            print('Authentication failed: ${e.message}');
    }
} catch (e) {
    print('Unexpected error: $e');
}
```

Error codes

`PlatformException.details` carries a structured payload of `{'code': <stable code>, 'description': <native description>}`. Use `TSAuthenticationErrorDetails.fromPlatformException()` to read it — it maps the code onto the `TSAuthenticationErrorCode` enum and falls back to `unknown` for values it does not recognize:

```
try {
    await auth.registerTOTP(uri, TSTOTPSecurityType.devicePin);
} on PlatformException catch (e) {
    final details = TSAuthenticationErrorDetails.fromPlatformException(e);
    if (details.code == TSAuthenticationErrorCode.devicePinNotAvailable) {
        print('Ask the user to set a device PIN first');
    } else {
        print('Registration failed: ${details.description}');
    }
}
```

Data Types

TSPinCodeRegistrationCompletion

```
class TSPinCodeRegistrationCompletion {
    final String publicKeyId;
    final String publicKey;
    final String keyType;
    final String contextIdentifier;
}
```

TSPinCodeAuthenticationCompletion

```
class TSPinCodeAuthenticationCompletion {
    final String publicKeyId;
    final String signature;
    final String challenge;
}
```

TSBiometricsRegistrationResult

```
class TSBiometricsRegistrationResult {
    final String publicKey;
    final String publicKeyId;
    final String os;
    final String keyType;
    final String? attestation; // Biometric attestation data (optional)
}
```

TSBiometricsAuthenticationResult

```
class TSBiometricsAuthenticationResult {
    final String publicKeyId;
    final String signature;
}
```

TSNativeBiometricsUnregisterResult

```
class TSTNativeBiometricsUnregisterResult {  
    final String publicKeyId;    // Public key ID of the unregistered biometric  
}
```

DeviceInfo

```
class DeviceInfo {  
    final String publicKeyId;  
    final String publicKey;  
}
```

TSTOTPSecurityType

```
enum TSTOTPSecurityType {  
    biometric,    // Requires biometric authentication to generate codes  
    none          // No additional security required  
}
```

TSTOTPRegistrationCompletion

```
class TSTOTPRegistrationCompletion {  
    final String? issuer;    // TOTP issuer (optional)  
    final String? label;    // TOTP label (optional)  
    final String uuid;       // Unique identifier for the TOTP  
}
```

TSTOTPGenerateCodeCompletion

```
class TSTOTPGenerateCodeCompletion {  
    final String code;       // Generated TOTP code  
}
```

Troubleshooting

Common Issues

1. iOS Build Errors

- Ensure iOS deployment target is 15.0+
- Module uses Swift Package Manager (SPM) - no CocoaPods setup needed

2. Android Build Errors

- Check minSdkVersion is 23+
- Verify all required permissions are added

3. R8/ProGuard Issues (Release Builds)

- The plugin includes consumer ProGuard rules automatically, in `android/consumer-rules.pro`. These cover the `com.transmit.authentication.** surface` **and** a narrow set of shared-core (`com.ts.coresdk.**`) crypto, error, logging and network classes,

which the plugin carries on core's behalf because neither the native Authentication SDK nor core-android-sdk ships consumer rules of its own.

- **This class of failure only reproduces in a release build.** A debug build runs with `minifyEnabled false`, so reflective and name-based serialization keeps are never exercised.
Test with `flutter build apk --release` (or your release variant) before shipping.
- If you still encounter minification issues, add these rules to your app's `android/app/proguard-rules.pro`:

```
# Keep TSAAuthentication SDK classes
-keep class com.transmit.authentication.** { *; }
-keep interface com.transmit.authentication.** { *; }

# Shared core, if you hit a stripped/renamed core class not covered
above
-keep class com.ts.coresdk.** { *; }
```

- Note: the plugin deliberately does **not** keep `com.ts.coresdk.device.**` or `com.ts.coresdk.geolocation.**`. No +A API reaches them, so keeping them would force unnecessary rules on every integrator. If you also use the Mosaic DRS or IDO SDKs, their own rules cover those packages.

4. New transitive dependency (core 1.0.30)

- Shared core 1.0.30 adds `com.google.android.gms:play-services-location:20.0.0`, which now arrives transitively through this plugin. If your app already depends on Play Services, check for a version conflict and align versions in your app's `build.gradle` if needed.

5. Biometric Not Working

- Check device has biometric hardware
- Verify permissions are granted
- Ensure user has enrolled biometrics

6. WebAuthn Not Supported

- Use `isWebAuthnSupported()` to check availability
- WebAuthn requires specific platform support

7. TOTP Issues

- Ensure TOTP URI format is correct: `otpauth://totp/...`
- Verify the UUID from registration is stored and used correctly
- For biometric security type, ensure biometrics are set up on device
- Check that TOTP registration completed successfully before generating codes

Debug Tips

- Enable verbose logging in debug builds
- Test on real devices for biometric features
- Check platform-specific error messages
- Verify network connectivity for WebAuthn

Support

For issues and questions:

- Check the troubleshooting section
- Review platform-specific requirements
- Test on multiple devices and OS versions